

Technical Interview Handbook

Distributed AI Job Orchestration Platform

Architecture, reliability, source navigation, and FAANG-style interview preparation

Source-verified against the local repository snapshot · 2026-10-07

Editable chapter source accompanies this PDF in `docs/interview-handbook/`.

Contents

1. Chapter 1 — Executive Overview
2. Chapter 2 — Problem Statement and Design Goals
3. Chapter 3 — Complete System Architecture
4. Chapter 4 — End-to-End Job Lifecycle
5. Chapter 5 — Transactional Outbox Deep Dive
6. Chapter 6 — Kafka and Asynchronous Processing
7. Chapter 7 — PostgreSQL and Data Consistency
8. Chapter 8 — Worker Reliability, Leases and Retries
9. Chapter 9 — Scheduling
10. Chapter 10 — AI Provider Integration
11. Chapter 11 — FastAPI and API Design
12. Chapter 12 — Frontend and User Experience
13. Chapter 13 — Authentication and Security
14. Chapter 14 — Observability and Operations
15. Chapter 15 — Testing and Quality Engineering
16. Chapter 16 — Technology Choices and Alternatives
17. Chapter 17 — Engineering Challenges and Trade-Offs
18. Chapter 18 — Scalability and Performance
19. Chapter 19 — Deployment Decision and Honest Limitations
20. Chapter 20 — Interview Question Bank with Answers
21. Chapter 21 — Mock Interviews
22. Chapter 22 — Source-Code Navigation Guide
23. Chapter 23 — Glossary and Final Revision Sheets

Chapter 1 — Executive Overview

What AetherFlow is

AetherFlow is a distributed job-orchestration platform for accepting bounded AI inference requests, persisting them, dispatching them asynchronously, executing them through a replaceable provider boundary, and letting an authenticated user inspect the durable lifecycle in a web console.

Orchestration is the control path around the work: validating the request, assigning an identity and idempotency key, recording intent, choosing when it may run, handing it to a worker, recording attempts and outcomes, retrying selected failures, and exposing status. It is deliberately more than calling an LLM endpoint.

The core persisted entities are users, jobs, idempotency records, attempts, results, lifecycle events, audit logs, refresh sessions, API keys, and outbox dispatches. PostgreSQL is authoritative. Kafka transports dispatch messages; it is not the job database. Redis is optional API rate-limit coordination, not a job cache, lock authority, or retry store.

Problem and intended users

An inference request may take much longer than an HTTP request should remain open. If the API waits synchronously, client timeouts, proxy limits, provider latency, and caller retries are coupled to execution. A durable asynchronous system instead acknowledges an accepted request after persistence, executes independently, and gives the caller a stable job identifier for later inspection.

The intended users are developers submitting structured inference workloads, operators inspecting job and dispatch behavior, and engineers studying reliability boundaries. Example real-world analogues include document classification, media processing, report generation, and offline enrichment. Those are examples of the pattern, not additional AetherFlow product features.

What distinguishes it from CRUD

A CRUD API saves and retrieves records. AetherFlow also has a versioned state machine, an outbox that couples database state to dispatch intent, an independent publisher, a Kafka consumer group, worker execution leases, bounded failure policy, durable retry intents, one-shot scheduling, and observable attempt/event history. Every boundary introduces a failure window that must be reasoned about. The project explicitly does **not** claim exactly-once execution.

Current capability and boundary

Implemented in the inspected source: authenticated registration and sessions, API keys, ownership checks, idempotent submission, PostgreSQL persistence, transactional outbox publication, Kafka dispatch, separate publisher/worker/scheduler processes, a deterministic mock provider, one OpenAI-compatible adapter, bounded execution and dispatch retries, expiring worker leases, one-shot schedules, cancellation, Redis rate limiting, metrics, and a Next.js console.

Not claimed: a public deployment, production readiness, high availability, provider-side deduplication, durable metrics, OpenTelemetry/Grafana, a worker registry, manual retry/replay, recurring schedules, email verification, password recovery, or performance benchmarks. See [docs/KNOWN-LIMITATIONS.md](#).

A 30-second explanation

AetherFlow is a local-first AI job orchestration project. The API validates and persists each job together with an outbox record in PostgreSQL. A separate publisher sends versioned dispatch messages to Kafka; workers claim them with state/version checks and an execution lease, call a provider adapter, and persist the result or a bounded retry decision. The Next.js console reads the durable state. Delivery is at least once, so I do not claim exactly-once provider execution or production deployment.

Use this as a structure, not a memorized personal story. Replace “I” with accurate ownership details for your own contribution.

A 2-minute explanation

The core problem is that inference can take longer than an API request should stay open, and client retries must not accidentally create duplicate jobs. Submission therefore validates a strict Pydantic request, authenticates the caller, scopes idempotency to that user, and writes a job, idempotency record, lifecycle event, and—when due—dispatch intent in one PostgreSQL transaction. A separate outbox process leases eligible intents, publishes them to Kafka, then records publication in another transaction. The unavoidable gap after Kafka acknowledges but before PostgreSQL finalization can create duplicate messages. The worker consumes with auto-commit disabled and acknowledges only after a durable decision. It checks message version and job state, claims `RUNNING` with an execution lease and attempt record, then calls a provider outside the database transaction. Success persists a normalized result and terminal state. Selected provider failures—rate limits, timeouts, and transient provider errors—can create a future outbox intent and `RETRY_SCHEDULED`; other failures or exhausted attempts become terminal. A separate database-backed scheduler activates one-shot due jobs. The console polls the API; it does not consume Kafka. This demonstrates practical failure reasoning, but tests use SQLite and fakes for most infrastructure boundaries. A limited Compose smoke path was observed locally; PostgreSQL concurrency and production operations are not proven.

A 5-minute walkthrough

1. **Start with the invariant.** A job row is the source of truth. API clients never set state directly; domain transitions use an allowed transition and a version-checked update.
2. **Explain submission.** `POST /api/v1/jobs` accepts an `Idempotency-Key`.
The request fingerprint includes the normalized request. A repeated key with the same fingerprint returns the existing job; a different payload conflicts.
3. **Explain the dual-write boundary.** The initial job and outbox intent are committed together. Kafka publication is separate, leased, and retrievable. This avoids losing dispatch intent if the API crashes after commit, but permits duplicate publication after Kafka acknowledgement.
4. **Explain worker ownership.** A Kafka message contains a job ID and version, not an authoritative copy of job state. The worker reloads the row, checks eligibility, records `RUNNING`, lease owner/expiry, and an attempt, and calls a normalized executor outside the transaction.
5. **Explain failure.** Only three categories are retried by the current execution policy. A retry transition and future outbox row are atomic. Crashed executions are not resumed; an expired lease is resolved to `SUCCEEDED` if a durable result exists or `FAILED` otherwise.
6. **Explain scheduling and UI.** Future `schedule_at` jobs are activated by a bounded PostgreSQL poller. The UI reads status, attempts, events, and results through the API and polls while a detail page is active.
7. **Close honestly.** Cite the tests and the limited local smoke verification, then state the unverified dimensions: PostgreSQL lock races, broker rebalancing, restart recovery at scale, and production deployment.

Chapter 2 — Problem Statement and Design Goals

Requirements that motivate the pattern

Concern	Why it matters	AetherFlow behavior
Long-running work	Provider execution may outlive an HTTP request budget.	API returns a persisted job; worker executes independently.
API responsiveness	The submit path should not hold a client request open for inference.	Submission commits and returns before provider execution.
Durable state	A process restart must not erase accepted work or results.	PostgreSQL stores job, attempt, result, event, and dispatch intent.
Event delivery	A committed job must eventually become eligible for dispatch.	Transactional outbox plus independent publisher.
Failure classes	Some errors may improve with retry; others will not.	<code>ExecutionFailureKind</code> and a bounded retry policy.
Scheduling	Future execution must survive restarts.	UTC <code>schedule_at</code> row and database poller.
Crash recovery	A <code>RUNNING</code> row cannot remain ambiguous forever.	Expiring execution lease and explicit stale-execution outcome.
Visibility	A caller/operator needs evidence of current state and history.	Job, event, attempt, admin, readiness, and metric endpoints.
Access control	A user must not inspect another user's jobs.	Database-backed ownership filtering and role dependencies.

The first column describes general distributed-system motivations. The second column is grounded in the current implementation. The project does not implement arbitrary task plugins, recurring schedules, multi-tenant quotas, operator replays, or horizontal autoscaling policy.

Goals and non-goals

Goals: preserve durable submission; make transport failure recoverable; bound retry; make state transitions explicit; expose ownership-safe status; support a credential-free deterministic local demonstration; keep provider details outside domain state changes.

Non-goals: distributed transactions across PostgreSQL and Kafka; exactly once provider effects; immediate push updates; guaranteed cancellation of in-flight remote calls; production SLOs; performance claims without a reproducible benchmark.

Requirement questions to ask

- What is the job's idempotency scope and retention policy?
- Is the outcome safe to repeat, or does it need provider-side idempotency?
- What is the maximum attempt count and what failures are retryable?
- Does cancellation mean “do not start” or “abort remote work”?
- What is acceptable schedule lateness, and which clock is authoritative?
- What are the data-retention and sensitive-input rules?

Those are product/system decisions, not properties a message broker supplies automatically.

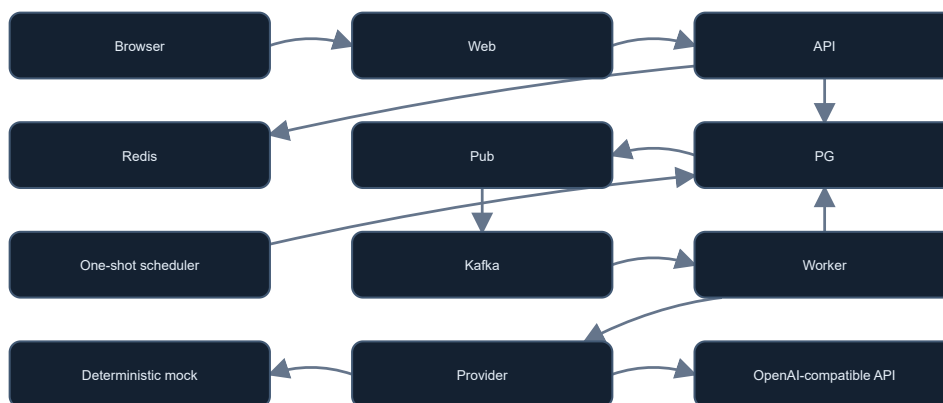
Chapter 3 — Complete System Architecture

Component responsibilities

Component	Responsibility	Must not become
Next.js console	Registration/sign-in, job submission/list/detail, admin inspection; calls API.	A source of authorization or job-state truth.
FastAPI API	Request validation, authentication, policy dependencies, query/submission services, HTTP error contract.	A synchronous provider runner or direct Kafka publisher.
PostgreSQL	Authoritative identity, state, attempts, results, lifecycle/audit records, outbox intents.	A place where the browser writes state directly.
Outbox publisher	Lease eligible rows, publish versioned messages, persist publication/failure metadata.	A state-owning worker or API request subroutine.
Kafka	Durable asynchronous dispatch transport and consumer-group coordination.	Job/result state storage or proof of provider completion.
Worker	Validate message against persisted state, claim execution, call executor, persist outcome, commit offset after durable decision.	A provider-specific business layer.
Scheduler	Find due ACCEPTED rows and atomically transition/create outbox intent.	A Kafka publisher or in-memory timer registry.
Provider adapter	Resolve provider, normalize outcomes and errors; mock or OpenAI-compatible implementation.	A database state-transition service.
Redis	Optional fixed-window API rate-limit counters.	A job cache, lock authority, idempotency store, or retry queue.

Processes are separate entypoints from a shared Python backend package. This is a modular monolith with process isolation, not a set of independently developed microservices.

System context



Request and message paths

- Browser read/write:** Next.js uses `NEXT_PUBLIC_API_BASE_URL`, a bearer token held in memory, and credentialed requests for the `HttpOnly` refresh cookie. It polls the API; it does not read Kafka or PostgreSQL.
- Submission:** API persists the job and idempotency record, and creates

outbox intent in the same transaction for immediate/past-due work. Future jobs wait as `ACCEPTED` until activation.

- **Publication:** publisher commits a short lease, performs Kafka I/O outside the database transaction, then finalizes publication in a separate transaction.
- **Execution:** worker consumes one record at a time in its loop, processes through `Worker`, and commits the Kafka offset after the durable decision.
- **Scheduling:** scheduler changes a due `ACCEPTED` job to `QUEUED`, records an activation event, and inserts an outbox row in one transaction.

Local topology

Compose starts PostgreSQL 16, a single-node Kafka 3.9 KRaft broker, Redis 7.4, one-shot Alembic migration, API, worker, outbox publisher, and scheduler. The Next.js development server runs outside Compose. Compose health checks cover PostgreSQL, Kafka, Redis, and API readiness. There are no dedicated worker/publisher/scheduler health endpoints. This topology is for reproducible local development, not an HA production design.

Boundary decisions

PostgreSQL is the source of truth; messages carry a versioned job reference. The scheduler writes database intent but never publishes directly. Domain services own transition rules. Provider adapters return normalized `ExecutionOutcome/ExecutionFailure` values. Redis outages bypass limits but do not affect job correctness. The key diagram and further flow details live in `docs/DATA-FLOW.md`.

Chapter 4 — End-to-End Job Lifecycle

Submission contract

The route is `POST /api/v1/jobs`; the request model is `JobCreateRequest` in `backend/src/aetherflow/jobs/schemas.py`. It forbids unknown fields, bounds `type/model/priority/timeout/retry-policy` fields, requires `schedule_at` to be timezone-aware, normalizes it to UTC, and rejects credential-like configuration keys. `input`, `configuration`, and `metadata` are dictionaries; that does not make arbitrary tools or code executable.

Example request:

```
POST /api/v1/jobs
Authorization: Bearer <short-lived-access-token>
Idempotency-Key: interview-demo-001
Content-Type: application/json
```

```
{
  "type": "structured_inference",
  "model": "mock-model",
  "input": {"prompt": "Classify this short sample."},
  "configuration": {"provider": "mock"},
  "priority": 0,
  "timeout_seconds": 30,
  "retry_policy": {
    "max_attempts": 2,
    "initial_backoff_seconds": 1,
    "max_backoff_seconds": 5,
    "backoff_multiplier": 2,
    "jitter": false
  },
  "metadata": {}
}
```

This is an illustrative shape matching the Pydantic schema, not a captured response or a promise that every provider accepts every model name. The mock accepts any non-blank model identifier.

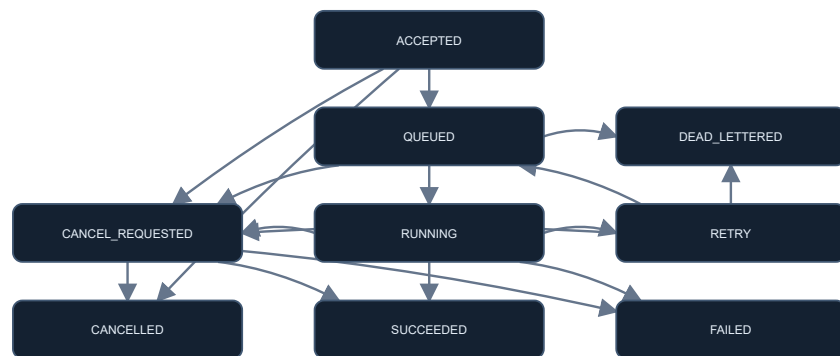
Step-by-step state/data path

1. **Authenticate and authorize.** The route requires a non-demo authenticated user. Authentication loads current user roles; ordinary job lookup is owner-scoped unless the user is an authorized administrator.
2. **Validate idempotency.** `validate_idempotency_key` checks the header; `submit_job` fingerprints the Pydantic JSON representation and scopes the unique record to the principal.
3. **Build durable records.** New `Job` starts at `ACCEPTED`, version 1. `JOB_ACCEPTED`, `IdempotencyRecord`, and an initial `OutboxDispatch(message_type="JOB_DISPATCH", schema_version="v1")` are staged in the same SQLAlchemy session.
4. **Commit.** The API returns HTTP 201 for creation. A matching retry returns the saved job with HTTP 200. Reusing a key with different input returns a conflict. A unique-index race is handled by reloading the committed record and comparing its fingerprint.
5. **Publish.** Outbox publisher lease → Kafka producer `send_and_wait` with job UUID key and an `aetherflow.job-dispatch.v1` envelope → separate `PUBLISHED` update. Transient publication failure records bounded metadata and next-attempt time; invalid records become `PERMANENT_FAILURE`.
6. **Claim.** Worker reloads the job, rejects terminal/stale versions,

transitions eligible work through `QUEUED` to `RUNNING`, stores an execution owner/expiry, and commits an attempt before provider invocation.

7. **Execute.** The provider call runs outside the database transaction under `asyncio.wait_for` using the job timeout.
8. **Persist.** A success stores attempt outcome, unique result, clears lease, reloads state, and transitions to `SUCCEEDED`. These are separate commits; lease recovery uses the presence of a durable result if the process stops in between.
9. **Retry or fail.** A retryable failure within budget atomically commits attempt metadata, `RUNNING` -> `RETRY_SCHEDULED`, lease clearing, and a future-dated versioned outbox intent. Otherwise it transitions to `FAILED`.
10. **Observe.** `GET /api/v1/jobs/{id}`, `/events`, and `/attempts` return owner-scoped durable data. Browser detail polling is bounded at a 10 second interval while visible and stops at terminal state.

State machine



`FAILED`, `SUCCEEDED`, `CANCELLED`, and `DEAD_LETTERED` are terminal. In the current local working tree, a scheduled job still `ACCEPTED` is cancelled directly to `CANCELLED`; an immediate job can use `CANCEL_REQUESTED`. A pending Kafka message for a now-terminal scheduled job becomes an ineligible no-op. This behavior is verified by the current cancellation service tests; older ADR wording predates the current change.

Chapter 5 — Transactional Outbox Deep Dive

The dual-write problem

Suppose an API performs two independent writes:

1. Commit a job row to PostgreSQL.
2. Publish a Kafka message.

If it commits first and crashes before publish, a durable job is stranded. If it publishes first and the database transaction fails, a worker can receive a job that the API reports as nonexistent. Retrying either operation can also create duplicates. There is no ordinary local transaction that atomically commits both PostgreSQL and Kafka.

AetherFlow's approach and transactions

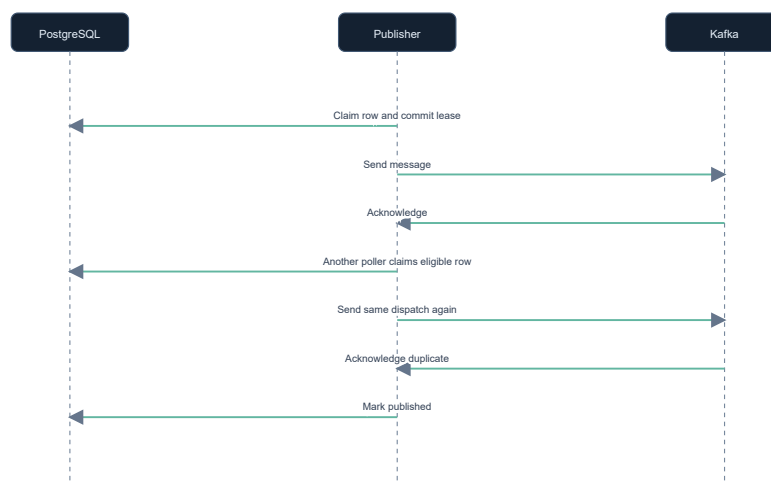
T1 — submission transaction: create `jobs`, `job_events`, `idempotency_records`, and an initial `outbox_dispatches` row together. Commit. The row's immutable dispatch fields are job ID/version, schema, and enqueue time. A future-dated job deliberately has no initial intent until activation.

T2 — publisher claim transaction: select one eligible outbox row with `FOR UPDATE SKIP LOCKED` where supported; write publisher owner, lease expiry, and increment attempt count. Commit the lease before network I/O.

Network step: produce the versioned message to Kafka and wait for `send_and_wait` acknowledgement using `acks="all"`. Producer idempotence is enabled by default but configurable.

T3 — finalize transaction: update the outbox row to `PUBLISHED`, clear its lease/error metadata. A publish failure instead records failure category and backoff or terminal `PERMANENT_FAILURE`.

Crash timeline



The finalization failure after a Kafka acknowledgement is the unavoidable duplicate-publication window. Conversely, failure before Kafka accepts the message is retrievable from the retained outbox row. Lease ownership predicates prevent one publisher from finalizing another publisher's claim; expiry makes abandoned claims recoverable.

Guarantees and non-guarantees

- **Guaranteed by the transaction boundary:** if the job transaction commits, the immediate dispatch intent commits with it. A future schedule is durable without needing an in-memory timer.

- **Not guaranteed:** one and only one Kafka record; one and only one provider invocation; atomic commit across PostgreSQL and Kafka; operator replay of a permanent record.
- **Duplicate safety:** the worker compares message job version to persisted job version, checks current state, and uses compare-and-set transitions. Unique attempt/result constraints protect durable records. They cannot undo an already-performed external inference call.
- **Idempotency is separate:** HTTP idempotency prevents duplicate job creation for the same principal/key/fingerprint. It is not the same as Kafka producer idempotence, consumer offset commits, or provider-side idempotency.

Kafka "exactly once" features, when available, cannot extend a transaction through a third-party LLM call and PostgreSQL result commit. The defensible description is at-least-once dispatch with state/version-guarded durable processing and possible repeated external work.

Operational trade-offs

The outbox adds a table, leases, polling, indexes, bounded retry metadata, and an operational backlog that must be monitored. In exchange, API latency and availability are less directly coupled to Kafka. There is currently no UI or API to replay permanent failures; a production design would need an audited, version-safe replay policy, retention, and poison-message handling.

Source references: `backend/src/aetherflow/jobs/service.py`, `backend/src/aetherflow/jobs/outbox.py`, `backend/src/aetherflow/infrastructure/kafka.py`, `backend/src/aetherflow/infrastructure/database/models.py`, `backend/tests/test_outbox.py`, ADR-0012 and ADR-0013.

Chapter 6 — Kafka and Asynchronous Processing

The vocabulary

- A **topic** is a named append-only stream of records. AetherFlow's dispatch topic is configured as `aetherflow.jobs` by default.
- A **partition** is an ordered log within a topic. Ordering is per partition, not global. The producer keys a record by job UUID, which keeps a job's records on the same partition under Kafka's normal partitioner.
- A **producer** serializes and sends dispatch records. The adapter awaits `send_and_wait`; default `acks="all"` asks the in-sync replica set to acknowledge, but local Compose has a single broker.
- A **consumer group** divides partitions among consumers in that group. A partition is assigned to at most one active group member at a time; adding workers beyond partition count does not add useful active partition parallelism.
- An **offset** is a record position. AetherFlow disables auto-commit and commits the received offset only after the worker's processing decision.

The envelope is versioned JSON (`aetherflow.job-dispatch.v1`) containing a job ID, job version, enqueue timestamp, and schema version. The Kafka record is a dispatch notification, not the complete job or its result.

Delivery and ordering

If the process fails before committing the Kafka offset, Kafka can deliver the record again. The worker's persisted state/version checks make many duplicate or stale records durable no-ops; they do not make provider side effects exactly once. A new retry is represented by a new job version and a new outbox record with a due time. Kafka order alone does not enforce lifecycle order across PostgreSQL commits, multiple dispatches, or retry timers.

Default producer idempotence reduces duplicate records generated by producer retries within Kafka's supported producer session semantics. It cannot close the separate database-finalization gap in the outbox, nor deduplicate a provider request after a worker crash.

Unavailable broker, backpressure, and diagnosis

When Kafka is unavailable, the API can still durably accept immediate work while PostgreSQL is healthy: the outbox intent remains the publication boundary. The publisher cannot complete delivery; it records retrievable failures where it can reach PostgreSQL, and the queue of unpublished intents can grow. When Kafka is reachable again, the publisher can retry due rows. Readiness checks PostgreSQL, not Kafka, so API readiness is not proof that dispatch is flowing.

Backpressure currently comes from the bounded outbox publication batch, publisher polling, Kafka partitions, worker process count, provider latency, timeouts, and database pool. The worker loop processes records sequentially per consumer instance; adding a consumer only helps up to available partitions and safe DB/provider capacity. The repo does not claim a tuned throughput limit.

For a backlog: compare unpublished outbox count/oldest age and dispatch failures with Kafka connectivity and lag; check scheduler due count; then inspect worker logs, active leases, attempts, provider failures, and database availability. Metrics are process-local, so an external scraper must collect each live process for operational use.

Why Kafka, and what else could fit?

Kafka is useful when a durable replayable partitioned stream and consumer groups justify an independently operated broker. The repo records Kafka dispatch intent in ADR-0003/0005/0012 and deployment rationale in ADR-0017; do not claim unrecorded alternatives were part of the original selection.

Alternative	Benefits	Costs / when it may fit better
PostgreSQL polling queue	Fewer moving parts; transactional claim and job data in one system; reasonable for modest workloads.	DB polling and row contention can become coupled to application reads/writes; choose when broker scale/replay is not needed.
Redis-backed queue	Familiar fast enqueue and delayed jobs; useful if Redis queue semantics already fit operations.	Redis persistence/HA and job semantics must be configured; AetherFlow's Redis is only optional rate limiting.
RabbitMQ	Routing, acknowledgements, work-queue semantics; suitable for command/task delivery.	Different operational/throughput model; adds another broker boundary.
Managed cloud queue	Less broker operations for a single cloud deployment; dead-letter and visibility-timeout features may help.	Cloud coupling, quotas/cost, and less portable local parity.
Direct synchronous provider call	Simplest when tasks are short, low-volume, and retry semantics are unnecessary.	Couples caller availability/latency to provider and loses durable asynchronous lifecycle.

Interview answer: Kafka is a documented dispatch design choice, not proof that Kafka is universally superior. If measured traffic and replay needs did not justify a separate broker, a PostgreSQL queue might be simpler; that would be a legitimate architecture, not an inferior one by default.

Chapter 7 — PostgreSQL and Data Consistency

Actual persistent model

Table	Purpose
users, roles, user_roles	Identity and current database-backed role membership.
refresh_sessions, api_keys, audit_logs	Rotating refresh-session family, hashed API credential material, and administrative/security audit records.
jobs	Owner, input/configuration, state/version, schedule, execution lease, retry policy and lifecycle timestamps.
idempotency_records	Principal/key uniqueness, request fingerprint and original job.
job_events	Append-style state and lifecycle audit history.
job_attempts	Numbered execution attempts, worker/provider/error/usage/trace metadata.
job_results	One normalized result row per job.
outbox_dispatches	Versioned dispatch intent, publication state, lease, attempts and retry timing.

The SQLAlchemy models are the current object mapping; Alembic revisions are the schema history. Migration `0001_identity_access` establishes identity, sessions, keys and audit tables. Later revisions add jobs/retries, `0006_durable_scheduling` adds `schedule_at` and due indexes, and `0007_demo_role` adds demo-role support. The migration chain, not `create_all`, is the reproducible schema contract.

Important constraints/indexes include unique user email, role name, refresh token hash, API-key public ID, (`user_id`, `idempotency_key`), per-job attempt number, one result per job, and dispatch uniqueness by job/version. Indexes support owner/state listing, active leases, due schedules, outbox eligibility, session expiry, and lookup by public credential ID. See each migration for the authoritative DDL.

Transaction and concurrency boundaries

- Submission commits the job, idempotency record, event, and immediate outbox intent as one unit.
- Scheduler activation commits compare-and-set state/version change, event, and outbox intent as one unit.
- Execution failure plus `RETRY_SCHEDULED` and its next dispatch intent is committed atomically.
- Provider I/O occurs outside database transactions.
- Outbox Kafka I/O occurs outside the short claim/finalization transactions.
- Result, attempt completion, and final success transition are not one database transaction; lease recovery reconciles a durable result with `SUCCEEDED`, otherwise an expired execution becomes `FAILED`.

Optimistic version predicates and row locks protect lifecycle transitions. Unique constraints close duplicate-record races. PostgreSQL `SKIP LOCKED` is used for bounded claim work where supported; current SQLite unit tests do not prove its production concurrency behavior.

The async SQLAlchemy engine uses a per-process pool (`pool_size` default 5, `max_overflow` default 10, with configured timeout/recycle). A connection budget is approximately the sum of API, publisher, scheduler, and worker process pools across replicas, plus migrations/admin tasks. Scaling processes without budgeting connections can exhaust PostgreSQL

even when each pool looks small.

Evolution and larger-scale options

Additive migrations should precede code that needs new fields; remove/rename only after old code is gone. Backfills need bounded batches and an explicit rollback/recovery plan. Large tables may need retention/partitioning for events, attempts, audit and outbox; hot job rows may need separate read models or archival. Do not split data prematurely: PostgreSQL currently simplifies the consistency boundary that motivates the outbox.

Isolation questions

At read-committed isolation, two concurrent actors can both observe an eligible state. Correctness therefore depends on row locks, unique constraints, lease-owner predicates and compare-and-set version updates—not the assumption that a read remains current. Test actual PostgreSQL lock races, deadlocks and lease expiry before claiming concurrency guarantees.

Chapter 8 — Worker Reliability, Leases and Retries

Claim and execute

For an eligible dispatch, the worker reloads the job and compares message version/state with persisted state. It records `RUNNING`, increments state version, sets `execution_owner` and `execution_lease_until`, and records the attempt before calling the executor. State transitions are guarded by both expected state and version. Provider code receives an immutable `ExecutionRequest`; it does not hold a database transaction open.

Leases bound how long a worker's claim is considered live. A lease is not a remote cancellation primitive and does not fence a provider that ignores cancellation. State/version checks fence stale durable writes after another actor changes the row, but cannot reverse an external side effect already performed.

Retry policy as implemented

Current execution retries are limited to `RATE_LIMIT`, `TIMEOUT`, and `TRANSIENT_PROVIDER`. Backoff is bounded and deterministic (capped exponential-style timing); configured `jitter` is accepted but not applied. The retry budget comes from the job request. When exhausted, the job becomes `FAILED`; retryable attempts otherwise atomically record the attempt result, transition to `RETRY_SCHEDULED`, and create the future outbox row.

`UNEXPECTED` is a known discrepancy: ADR-0015 describes unexpected internal failures among retry candidates, but the current retryable-category set does not include it. Explain current source behavior, not the older intention. Do not silently call every exception retryable. The separate outbox publisher also has its own bounded dispatch retry policy; it is not the execution retry counter.

Crash and recovery cases

Failure window	Durable evidence	Current recovery / residual risk
Before execution claim commit	Job not durably <code>RUNNING</code> ; Kafka offset not committed on unhandled processing failure.	Record can be redelivered; claim is attempted again.
After claim, before provider call	<code>RUNNING</code> , attempt and lease exist.	Stale lease scan resolves to <code>FAILED</code> if no result; work is not resumed.
Provider performed work, worker died before result commit	Lease/attempt exist; external service may have acted.	Recovery cannot know provider effect; retry/re-delivery can repeat the external request.
Result committed, final transition missing	Durable unique result plus nonterminal job may remain.	Expired-lease recovery chooses <code>SUCCEEDED</code> when result exists.
Failure finalized with retry intent	<code>RETRY_SCHEDULED</code> and future outbox row commit together.	Publisher sends when eligible; duplicate dispatch is state/version checked.
Cancellation races an in-flight provider call	<code>CANCEL_REQUESTED</code> may coexist while call runs.	Completion may still end <code>SUCCEEDED</code> or <code>FAILED</code> ; cancellation cannot promise remote abort.

The precise stale recovery result is intentionally explicit: expired work is not re-executed automatically; an existing result reconciles to success, absence of a result resolves to failure. This preserves a durable terminal state while honestly accepting possible lost/duplicated provider work.

Graceful shutdown and stuck-job diagnosis

The worker is a long-running async process and Kafka auto-commit is disabled. Do not claim that shutdown safely cancels every provider request: the provider may already be executing remotely. A production shutdown protocol should stop fetching new messages, bound the drain period, leave unfinished offsets uncommitted, and let lease recovery reconcile the claim.

For a job stuck in `QUEUED`, inspect the matching outbox row, publish state, Kafka connectivity/consumer lag, then worker availability. For `RUNNING`, inspect owner/lease expiry, attempt start/completion, timeout, result presence, and worker logs. For repeated `RETRY_SCHEDULED`, inspect failure kind, attempt count, `retry_at`, provider health and rate limits. Correlate by job ID, request ID and trace ID; redact prompt/credential material.

Chapter 9 — Scheduling

Stored schedule and due activation

A one-shot schedule is `jobs.schedule_at`, a timezone-aware timestamp normalized to UTC. A future submission remains `ACCEPTED` and has no dispatch intent. A past or immediate submission follows ordinary dispatch. Migration `0006_durable_scheduling` adds a direct schedule index and a composite due index (`state, schedule_at, created_at`).

The scheduler polls due `ACCEPTED` records in a bounded batch. It uses PostgreSQL row claiming/locking and a state/version compare-and-set to change the job to `QUEUED`, write a lifecycle event, and create an outbox dispatch in one transaction. It then yields transport publication to the outbox publisher. A concurrent scheduler should skip locked work or fail its compare-and-set rather than activating the same current version twice; the database uniqueness and state/version conditions are the safety boundary. Multi-process PostgreSQL behavior remains unverified by SQLite tests.

The scheduler uses UTC instants rather than local wall-clock interpretation. This avoids ambiguous local times at DST boundaries; API validation rejects naive timestamps. Clock skew still matters for due cutoffs and leases. Production replicas should use synchronized clocks and monitor schedule lateness. Poll interval and batch size trade dispatch latency against database poll load.

Cancellation and restart

In the current working tree, cancelling a scheduled job while it remains `ACCEPTED` changes it directly to `CANCELLED`; it never enters the scheduler due query. If activation wins the race and state becomes `QUEUED`, cancellation uses the ordinary cancellation path. A stale already-published dispatch is checked against current state and becomes a no-op.

Schedules survive scheduler restart because they live in PostgreSQL. The restart process resumes polling due rows; no in-memory timer is required. Current tests cover UTC normalization, due activation, duplicate polling, cancellation, and idempotency conflicts, but process restart and real PostgreSQL races are not established.

Only one-shot timestamps are implemented. Cron/recurrence would need a schedule definition separate from each execution, timezone rules, missed-run policy, per-occurrence idempotency, and bounded catch-up semantics. A managed workflow scheduler or database-backed poller may be preferable depending on complexity; a separate scheduler service is justified only if load or operational isolation requires it.

Chapter 10 — AI Provider Integration

Boundary and data contract

`JobExecutor` in `jobs/execution.py` accepts an immutable execution request and returns an `ExecutionOutcome` or raises normalized `ExecutionFailure`. The resolver in `jobs/providers.py` selects the configured adapter. The deterministic mock is the default and makes local tests/demos independent of network access and API secrets. The OpenAI-compatible adapter uses HTTPX and normalizes HTTP/provider failures and timeouts.

The accepted job input/configuration is Pydantic-validated at the API boundary. The provider outcome is normalized as a dictionary plus optional usage, `schema_version`, provider and finish reason. That is not a job-type-specific typed output schema or proof that every arbitrary output conforms to a customer JSON Schema. Do not describe it as stronger validation than it is. Input configuration that looks like credentials is rejected; credentials belong in runtime environment/settings, not a job payload.

There is no configured automatic multi-provider fallback in current source. The worker retries selected normalized transient classes, but a different provider is not silently selected as a fallback. An external provider brings latency, availability, cost, quota, data-residency and output-drift risks. Never include API keys in logs, job config, screenshots, or interview demos.

Choosing the boundary

Provider-specific SDK calls directly in the job service would couple business state transitions and tests to one vendor. An adapter makes deterministic mocking and failure normalization simpler. Its costs are an interface to maintain and the possibility of pretending different providers have identical capabilities. An interface should expose only the common contract and retain provider-specific metadata where useful.

Retrospective improvement: define explicit, versioned per-job output schemas, validate before persistence, classify malformed output separately, and add a provider idempotency key if the vendor can honor it. Keep raw provider response retention and prompt logging constrained by privacy policy. No live external-provider request or credential was used in the recorded verification.

Source references: `backend/src/aetherflow/jobs/execution.py`, `providers.py`, `openai.py`, `jobs/worker.py`, and `backend/tests/test_phase5_providers.py` / `backend/tests/test_phase9_provider.py`.

Chapter 11 — FastAPI and API Design

Routing, validation, and dependency boundaries

The application composes versioned routers under `/api/v1`. Route functions are HTTP adapters: Pydantic request/response models define the wire contract; dependencies resolve the async SQLAlchemy session and authenticated principal; job/auth services enforce domain rules. This keeps provider execution out of the request handler. OpenAPI is generated from FastAPI route/schema declarations; health and metrics routes can be excluded from the public schema.

Unknown request fields are rejected on key public schemas. The error contract is normalized to:

```
{
  "error": {
    "code": "VALIDATION_ERROR",
    "message": "Request validation failed.",
    "request_id": "request-id",
    "details": {"fields": [{"location": ["body", "model"], "message": "Field required"}]}
  }
}
```

The exact example `request_id` is illustrative. Pydantic validation is mapped to HTTP 400. Expected domain/API errors preserve status and stable error code; unexpected exceptions are logged with server context and return a generic HTTP 500 body rather than a traceback.

Endpoint map

Method/path	Purpose	Access
POST <code>/api/v1/auth/register</code>	Create identity.	Public, subject to configured controls.
POST <code>/api/v1/auth/login</code>	Issue access token and refresh cookie.	Public, rate-limited.
POST <code>/api/v1/auth/refresh</code>	Rotate refresh session and issue a new access token.	Cookie + allowed origin.
POST <code>/api/v1/auth/logout</code>	Revoke session family and clear refresh cookie.	Cookie/origin-aware.
GET <code>/api/v1/auth/me</code>	Return current principal and roles.	Authenticated.
POST <code>/api/v1/jobs</code>	Idempotently submit a job.	Authenticated non-demo principal.
GET <code>/api/v1/jobs</code>	List caller's jobs with bounded pagination/filters.	Authenticated.
GET <code>/api/v1/jobs/{id}</code>	Read owner-scoped job/detail.	Owner or authorized admin.
POST <code>/api/v1/jobs/{id}/cancel</code>	Request/perform state-safe cancellation.	Owner or authorized admin.
GET <code>/api/v1/jobs/{id}/events</code>	Read lifecycle history.	Owner or authorized admin.
GET <code>/api/v1/jobs/{id}/attempts</code>	Read execution attempts.	Owner or authorized admin.
GET <code>/health/live</code> , GET <code>/health/ready</code>	Process liveness and database readiness.	Operational probes.
GET <code>/metrics</code>	Prometheus exposition.	Operational route.

Administrative routes under `/api/v1/admin` provide bounded queue/job and redacted operational views to admins. Check `backend/src/aetherflow/api/v1` for the exact filters and response model before quoting one. The implemented pagination for user event/attempt collections is limit/offset; do not call it cursor pagination.

Example response shape for a created job (values illustrative):

```
{
  "id": "11111111-1111-4111-8111-111111111111",
  "state": "ACCEPTED",
  "version": 1,
  "type": "structured_inference",
  "model": "mock-model",
  "created_at": "2026-01-01T12:00:00Z"
}
```

Use the actual response schema in `jobs/schemas.py` rather than promising these exact values or field subset. API versioning is a URL prefix; no claim of a multi-version migration policy is made.

Health, readiness, limiting and error behavior

Liveness is dependency-free. Readiness probes PostgreSQL and returns unavailable if the database check fails; it does not test Kafka, Redis, or an external inference provider. API rate limiting is middleware around `/api/v1/` routes with a separate auth class; it is optional and Redis fail-open. This is a deliberate availability/abuse-control trade-off, not a strict security boundary.

Design alternatives

FastAPI gives typed request validation and OpenAPI with async endpoint support. Django REST Framework could be preferable for a product needing its integrated admin/ORM/auth ecosystem; Flask could be adequate for a smaller synchronous API but requires assembling more conventions. These are retrospective comparisons: no repository evidence says they were all evaluated during original selection.

Chapter 12 — Frontend and User Experience

Actual UI structure and API interaction

The frontend is a Next.js App Router application in `frontend/app`, React 19, TypeScript, CSS/Tailwind-based styling, Lucide icons, and a small set of product-specific components. The console offers sign-in/registration, an optional read-only demo mode, job submission/list/detail, filters and pagination, cancellation confirmation, and admin operational views. Do not describe it as a real-time WebSocket console: job detail polls the API about every 10 seconds while the document is visible and the job is nonterminal.

`frontend/lib/api/client.ts` centralizes the API base URL, bearer token in memory, credentialed fetches for refresh cookies, structured `ApiError` responses, bounded query parameters, and a refresh attempt after an eligible

1. `auth-provider.tsx` coordinates registration/login and session

refresh. The UI avoids putting long-lived bearer credentials in local storage; refresh is handled with an `HttpOnly` cookie. A stale return URL is constrained to safe in-app destinations.

Job detail presents current state and result, event/attempt histories, and a cancellation flow. Polling is suspended while hidden and resumes when the document becomes visible. The view distinguishes an accepted scheduled-job cancellation from a general cancellation request. Loading, errors, retry, terminal-state and no-data surfaces are covered by component/API-client tests.

Frontend alternatives

Next.js supports server/client composition and routing; a Vite SPA could be simpler if server rendering and framework routing are not needed. A heavier design system could improve accessibility consistency at the cost of dependencies and overrides. Current implementation uses custom console components; do not infer `shadcn/ui` or `Framer Motion` use from the original product brief. Alternatives here are retrospective rather than documented original selection history.

UX trade-offs

Polling is easy to reason about and recovers after tab re-entry, but it increases periodic read traffic and can delay updates. At higher scale, server-sent events or WebSockets could lower perceived latency, but require connection management, authorization on streams, fan-out/backpressure, and a durable catch-up path. Terminal state still needs to be read from the API.

Chapter 13 — Authentication and Security

Implemented controls

- **Passwords:** Argon2 `PasswordHasher`; request schema bounds password length and rejects blank values.
- **Access:** short-lived HS256 bearer access tokens. Authentication rechecks the current user/session family and loads current database roles; a role is not trusted solely because an old browser screen displays it.
- **Refresh:** opaque random refresh material is stored hashed, set in an `HttpOnly` cookie, rotated on use, and bound to a session family. Logout revokes the family so associated access tokens become invalid under the implemented check.
- **API keys:** one-time secret display, public-ID lookup and stored hash; revoke and last-used metadata. Treat the full key like a password.
- **Authorization:** service queries constrain ordinary users to their own jobs; admin views require admin role. The optional DEMO principal is read-only and restricted from submission/mutation.
- **Browser origin:** configured origin allowlist is applied to cookie-auth routes. CORS is explicit; wildcard credentials are not an acceptable substitute for an origin allowlist.
- **Secret handling:** settings come from runtime configuration; payload schemas reject credential-like job config. No environment secret values belong in this handbook.

Authentication answers “who is this caller?” Authorization answers “may this caller perform this operation on this exact object?” Owner filtering is therefore important on reads, not just on mutation. The auth tests include cross-user isolation and role/ownership field injection checks.

Controls that are absent or incomplete

Email verification and password recovery are absent. There is no claim of MFA, SSO, WebAuthn, account lockout, or a public production threat review. Rate-limiter keys use a public API-key identifier when available and client host otherwise; bearer users without API keys are therefore grouped by network host for the general limiter. Redis failure is logged/metriced and fails open, so protection is reduced during that outage. The demo password is shared only by an explicitly provisioned operator and is not a production identity strategy.

Threat-model prompts and improvements

Protect against credential stuffing, refresh-token theft/replay, IDOR, overprivileged demo/admin access, prompt/PII leakage, API-key exfiltration, forged client IP behind proxy headers, CSRF on cookie routes, origin misconfiguration, dependency compromise and resource exhaustion. Production readiness would include TLS and trusted proxy policy, rate limiting with known outage behavior, secret rotation, backups, retention/deletion policy, audit review, dependency scanning, security headers, abuse controls and incident response.

Interview rule: state the concrete present control first; distinguish “implemented” from “must be configured” and from “would add.” Never suggest that a bearer token alone proves the caller owns a job.

Chapter 14 — Observability and Operations

Signals that exist

The backend exposes Prometheus-compatible process-local metrics for request classes, job transitions, worker execution, retry decisions, outbox publication, rate limits/Redis errors, and durations. Labels are bounded to avoid arbitrary job IDs. Structured logs include request/job correlation where available. Readiness means PostgreSQL is reachable; liveness means the API process can answer. These are not interchangeable.

Metrics are in-memory and reset on process restart. The repository does not bundle a Prometheus server, Grafana dashboards, alerting, an OpenTelemetry collector, durable traces, or worker registry. A local metric endpoint is instrumentation, not a complete observability deployment.

Triage runbook

Symptom	First evidence	Next checks
API will not start	API process log and Compose status.	Settings validation, port binding, PostgreSQL URL/network, import/runtime error. Never print secret environment values.
Database connection failure	<code>/health/ready</code> , DB logs, connection count.	Host/port/database, migration completion, credentials source, pool budget, TLS policy.
Migration failure	Migration process exit and Alembic revision.	Database target, current revision, failed DDL, backup/rollback plan. Do not run destructive downgrade on shared data casually.
Kafka unavailable	Publisher/worker logs; unpublished outbox age/count.	Broker listener advertised to container network, topic/bootstrap setting, broker health, consumer lag and offset.
Jobs stay ACCEPTED (immediate)	Job and outbox rows; job version.	Publisher lease/retry/error state, Kafka connectivity, batch/poll process. Scheduled future ACCEPTED can be expected; inspect <code>schedule_at</code> .
Jobs stay QUEUED	Outbox published state and Kafka lag.	Worker process/group, partition assignment, dispatch version/state.
Jobs stay RUNNING	Execution owner/lease, attempt timestamps and result existence.	Worker crash/timeout, provider duration, stale-lease recovery cadence, worker logs.
Repeated retries	Failure kind, attempt count, <code>retry_at</code> , provider metadata.	Rate limit/quota, provider status, timeout budget, deterministic backoff, misclassified failure.
Provider timeout	Attempt error class and configured timeout.	External latency/availability, worker connectivity, per-job timeout, avoid exposing prompt or secret.
Frontend cannot reach API	Browser network panel and API base URL.	<code>NEXT_PUBLIC_API_BASE_URL</code> , allowed origin/CORS, credentials, TLS/mixed content, API readiness.

These are diagnostic starting points, not claims that every failure produces a single distinctive metric. Local logs and row state remain important because metrics are process-local.

Chapter 15 — Testing and Quality Engineering

Test architecture

Backend tests use `pytest`, `async fixtures`, an isolated SQLite database, dependency overrides and fake provider/dispatch/Redis boundaries. The suite covers state transitions, idempotency, auth/ownership, job API, cancellation, outbox behavior, retry classification, providers, metrics, scheduling, admin, pagination and onboarding. This makes domain and API paths deterministic; SQLite does not implement PostgreSQL's `SKIP LOCKED` semantics.

Frontend tests use Vitest, Testing Library and jsdom. They cover the API client, token/refresh behavior, validation, routes/components, job detail, pagination, cancellation, admin behavior, demo/read-only labelling and safe navigation. Separate ESLint, TypeScript and production build checks validate static and packaging properties.

Compose includes a local PostgreSQL/Kafka/Redis path. The observed smoke check used a stack that was already running, not a fresh rebuild with all current working-tree changes. It exercised mock success, retry exhaustion and a due scheduled job through the infrastructure path. It did not verify broker rebalance, PostgreSQL lock races, multi-process recovery or production availability. OpenAI-compatible HTTP behavior is tested with mock transport; no live credential was used.

Observed verification (2026-10-07 workspace)

`docs/TESTING.md` records the most recent observed results for this workspace:

- Backend `pytest tests -q`: **150 passed**, with 1,144 `pytest-asyncio` deprecation warnings.
- Ruff lint, Ruff format check (82 files), and mypy (57 source files): passed.
- Frontend Vitest: **32 passed**; ESLint, TypeScript `--noEmit`, and production build: passed.
- `docker compose config --quiet` and `git diff --check`: passed.
- The current uncommitted GitHub Actions workflow has not had a hosted workflow run observed.

The selected local Python environment reported Python 3.14.3, Windows; these results do not establish every supported Python version. Check the present `docs/TESTING.md` and rerun commands before using these numbers in a future interview. Counts are snapshot evidence, not a permanent promise.

Testing gaps and strategy

Before production claims, add real PostgreSQL concurrency tests for row locking/leases and scheduler races; Kafka restart/rebalance/redelivery tests; cross-process Redis limiter tests; subprocess shutdown/recovery tests; and provider contract tests with a controlled fake HTTP server. Add property/ state-machine tests for legal transitions and fault injection at each commit/ack boundary. Run load tests with reproducible environment, workload, percentiles and resource metrics before making performance claims.

Chapter 16 — Technology Choices and Alternatives

This is a **retrospective** comparison unless an ADR below is explicitly cited. Do not say a candidate technology was considered during original development without repository history or a decision record supporting that statement.

Technology	Role and fit in this project	Benefits	Costs / alternatives and when switching is justified
FastAPI	Async HTTP API, Pydantic validation, OpenAPI; useful for typed API/service boundaries.	Concise routes, generated schema, async ecosystem.	Django REST Framework brings integrated admin/auth conventions; Flask is smaller but leaves more conventions to assemble. Retrospective options; switch only if product needs justify migration.
Next.js	React console with App Router and typed client.	Routing/build ecosystem and React components in one app.	Vite SPA is simpler when server rendering/framework routing is not useful. Not an evidenced original comparison.
PostgreSQL	System of record, transactions, locks, constraints, due/outbox polling.	Strong relational constraints and transaction semantics central to outbox consistency.	MongoDB could fit document-centric flexible data, but relational ownership, idempotency and transaction/query needs are natural here. ADRs support PostgreSQL as durable system of record; they do not establish a MongoDB bake-off.
Kafka	Versioned dispatch topic and consumer group.	Durable partitioned log, replay/consumer-group model, independent publisher/worker processes.	PostgreSQL queue is simpler at modest load; RabbitMQ/managed queues may fit command semantics/operations better. Kafka rationale exists in ADR-0003/0005/0012/0017; no claim that alternatives were all tested.
Redis	Optional fixed-window rate limiting only.	Shared atomic counter boundary across API processes.	PostgreSQL rate-limit table is simpler but writes hot rows; gateway/managed limiter shifts operations. ADR-0020 documents coordination and rate-limit intent. Redis is not authoritative for jobs.
Docker Compose	Local PostgreSQL/Kafka/Redis/API/publisher/scheduler/worker/migrations environment.	Reproducible local dependencies and explicit process topology.	Testcontainers can isolate test dependencies; a managed dev stack reduces local resource use. Compose is not a production orchestrator.
Pydantic	Request, settings and response validation/serialization.	Runtime boundary validation plus schema integration.	Dataclasses/attrs are lighter for internal values but need separate parsing; Pydantic is already part of FastAPI's contract.
SQLAlchemy async	ORM, sessions, async DB transactions/queries.	Relationships, explicit transactions, typed persistence boundary.	Direct SQL/asyncpg gives more query control and less ORM behavior; choose where measured complexity warrants it.
Alembic	Versioned relational schema migrations.	Reviewable ordered upgrades/downgrades tied to SQLAlchemy metadata.	Framework-native migration systems are reasonable if framework changes; raw DDL scripts can suit a tiny schema but lack richer migration conventions.
pytest / pytest-asyncio	Backend unit/API/service test runner.	Fixture ecosystem, parametrization, async testing.	unittest is built-in and adequate for smaller suites; pytest fixtures fit the existing suite.
Vitest / Testing Library	Frontend component and client tests in jsdom.	Fast Vite-aligned runner and user-facing interaction tests.	Jest is mature, but changing brings migration cost without evidence of a current limitation.
Provider abstraction	JobExecutor boundary, mock executor, OpenAI-compatible adapter.	Tests avoid live credentials; business lifecycle does not embed provider SDK calls.	Direct SDK is simpler for a single throwaway provider, but couples retries/contracts to vendor. Abstraction is worthwhile while normalized behavior is needed; avoid over-generalizing provider-specific features.

Alternatives considered versus alternatives worth considering

ADRs state concrete rationale for Kafka/dispatch, at-least-once processing, outbox, recovery, metrics, deployment and Redis. They are design intent, not proof every alternative was prototyped or load-tested. FastAPI and Next.js do not have a documented head-to-head evaluation in the inspected ADR set. Frame non-documented options as retrospective alternatives, not project history.

Chapter 17 — Engineering Challenges and Trade-Offs

Documented design complexity

Complexity	Why it matters	Existing approach	Residual risk / interviewer probe
PostgreSQL/Kafka dual write	Either system can accept while the other fails.	Transactional outbox; separate publisher leases/finalization.	Kafka ack before DB mark can duplicate. “How do you prove no lost intent?”
At-least-once delivery	Consumer may see a record again.	Offset after durable decision; state/version guards, unique attempt/result.	Provider side effect can repeat. “What is idempotent here and what is not?”
External call versus state	Provider result cannot join local DB transaction.	Call outside transaction; lease and reconciliation by durable result.	Crash after provider success but before result means ambiguity.
Scheduling race	Multiple pollers or cancellation can contend.	DB locks, due-state filter, compare-and-set; current scheduled cancellation directly terminal.	PostgreSQL race behavior not proved by SQLite suite.
Configuration surface	Multiple local processes depend on URLs, secrets, topic and provider settings.	Pydantic settings and Compose examples/health checks.	A healthy API does not prove worker/broker/provider configuration.
Test determinism	Live external services make CI flaky and expensive.	SQLite and fakes for default tests; limited local Compose smoke.	Fakes cannot prove broker/database concurrency behavior.
Observability scope	Per-process metrics disappear on restart.	Prometheus exposition and structured logs.	No collector, dashboards or alerts are bundled.

Current source-versus-design discrepancy

ADR-0015 describes `UNEXPECTED` internal failures as retry candidates, while the present `jobs/retry.py` retryable set contains only rate-limit, timeout, and transient-provider categories. The handbook reports current executable behavior and marks the ADR as older intent. No change to runtime retry behavior was made for this documentation task.

Older ADR text also differs from current behavior for cancellation of a scheduled job while still `ACCEPTED`: the current working tree transitions it directly to `CANCELLED`; ADR-0021 describes the earlier flow. Likewise, ADR-0003 language about lifecycle-event topics is older than the current dispatch-only Kafka message path. The source and current flow docs are the behavioral truth; ADRs explain prior design intent.

Historical facts versus hypothetical scenarios

The repository documents architecture decisions and test scope, not personal production incidents or a historical debugging diary. The outbox crash window, broker outage, stale worker and cancellation race described here are failure scenarios derived from code boundaries, not claims that the author personally experienced them. The observed audit items are the retry discrepancy, stale ADR wording, process-local metrics, and gaps explicitly listed in `docs/KNOWN-LIMITATIONS.md` / `docs/TESTING.md`.

A defensible lesson

The valuable lesson is to phrase guarantees at the correct boundary. An outbox makes database intent durable; it does not make cross-system delivery exactly once. A lease protects a database claim; it does not stop an external provider. A passing fake-backed test verifies domain behavior; it does not prove PostgreSQL locking. An interviewer is likely to probe exactly those distinctions.

Chapter 18 — Scalability and Performance

No reproducible throughput or latency benchmark is present. Do not invent jobs/second, p95, cost-per-job or concurrency claims. A benchmark needs the source revision, workload mix, input sizes, provider/mock choice, host, database/broker configuration, process counts, duration, warmup, percentiles, error rate and resource use.

Scaling dimensions

- **Workers:** increase consumer instances up to topic partition count and safe provider/database capacity. Current per-instance loop is sequential. Add bounded concurrency only after defining per-job ordering, database sessions, shutdown, provider quota and lease renewal semantics.
- **Kafka:** partitions permit parallelism and per-key ordering; changing partition count can remap keys. Size for consumers and throughput, then validate rebalance behavior and broker durability/replication.
- **PostgreSQL:** likely bottleneck candidates include hot job-row updates, outbox eligibility scans, growing event/attempt tables, indexes, and connection count. Use query plans, lock/wait metrics and realistic retention tests before denormalizing or splitting services.
- **Connections:** calculate API + worker + publisher + scheduler pools across all replicas. A process-replica scale-out can exhaust the database before CPU is saturated.
- **Backlog/backpressure:** measure oldest outbox age, Kafka lag, due-schedule lateness, active lease age, provider latency and retry rate. Bound producer batches and worker concurrency. A faster enqueue path that lets a backlog grow without limit is not safe scaling.
- **Retry storms:** deterministic capped delay can synchronize retries. Jitter is accepted but not applied in the present policy; implementing and testing jitter, retry-after hints, circuit breaking or per-provider concurrency is future work, not current behavior.
- **Caching/rate limiting:** Redis is only optional fixed-window request limiting. There is no result/job cache. Cache only if measured read load and invalidation semantics justify it.
- **Dead lettering:** outbox permanent failure and `DEAD_LETTERED` state exist as concepts, but there is no operator replay endpoint/UI. A future DLQ workflow needs authenticated review, reason, idempotency, audit and safe version regeneration.

Measurement plan

1. Define a representative request/output distribution and target objectives.
2. Benchmark API persistence with PostgreSQL and no provider calls.
3. Benchmark outbox/Kafka/worker independently and end-to-end.
4. Sweep partitions, worker count, pool size, batch size and poll intervals.
5. Inject broker/database/provider errors; measure recovery and duplicate effects, not just successful throughput.
6. Record bottlenecks and repeatable scripts; add only observed results to README.

Chapter 19 — Deployment Decision and Honest Limitations

What is and is not verified

The repository provides a reproducible local Compose topology and a Next.js development setup. A limited local smoke check was observed through PostgreSQL, Kafka, publisher, scheduler and worker for mock success, retry exhaustion and a due schedule. The stack pre-existed and was not freshly rebuilt against every current working-tree change. Tests and `docker compose config --quiet` passed in the recorded workspace. These facts do not equal a public production deployment, high availability, cloud integration, cross-process recovery guarantee or real-provider success.

The project owner has not confirmed why there is no verified public deployment. Do **not** attribute it to cost, secrets, time, or any personal reason unless that is true and the author confirms it. A responsible, non-invented answer is:

I can show the repository, the local Compose environment, the deterministic tests, and the parts of the infrastructure flow that were exercised. I do not have evidence of a public production deployment, so I do not present it as production-operated. If I deploy it publicly, I first need to decide how to protect provider credentials and demo access, provision managed stateful dependencies, set backups/retention and TLS, add external scraping/alerts, and validate restart and concurrency behavior. The actual reason I have not deployed it is [state your real reason here].

Public hosting is not a prerequisite for explaining engineering work. A repository with architecture diagrams, migrations, executable tests, a repeatable local setup and a recorded walkthrough can be credible evidence, provided deployment status is described accurately.

Why deployment is a real decision

Kafka/PostgreSQL/Redis can exceed a small free tier and free services may sleep, lose ephemeral data, enforce quotas, or lack reliable backups. Exposing a job endpoint can invite provider-cost abuse; a public demo also needs strict quotas, prompt/data privacy decisions, credential isolation and safe demo identity provisioning. A demo account is not a substitute for authenticating every operation.

Before responsible production exposure

- Define SLOs, supported workload and abuse limits.
- Move credentials into a managed secret store; rotate any exposed secret.
- Deploy PostgreSQL with backups, restore drills, migrations and connection limits; provision Kafka/managed queue with replication and retention.
- Terminate TLS; set trusted proxy/client-IP handling, CORS/origin, security headers and account recovery policy.
- Configure durable metrics/traces/logs, dashboards, alerts and runbooks.
- Test PostgreSQL lock races, Kafka rebalances, process termination, migration rollback, provider timeouts, retention/deletion and cost limits.
- Add operational controls for poison messages, permanent outbox rows and audited retries/replays.

Describe a public deployment only after it has been performed and its reliability/operations evidence exists.

Chapter 20 — Interview Question Bank with Answers

How to use the answers

Start with the spoken version; expand into the deeper explanation only when the interviewer asks. Keep ownership statements factual: fill in the work you personally did, and do not borrow an architecture decision as proof that you implemented every part. “Verify” names source to open before the interview. The avoid note flags common overclaims.

A. Recruiter screening and project summary

A1. What is AetherFlow?

Spoken: “It is a local-first asynchronous AI job orchestration project: jobs are persisted, dispatched through an outbox and Kafka, executed by workers through a provider boundary, and inspected in a web console.”

Deeper: PostgreSQL is authoritative; Kafka carries versioned dispatch references; Redis is only optional rate limiting. Its value is the lifecycle and failure-boundary engineering, not an AI chat UI. It is not a public production service or benchmark.

Likely follow-up: What happens between `POST /jobs` and a result? **Avoid:** “It handles millions of requests” or “it is production-proven.” **Verify:** `docs/ARCHITECTURE.md`, `docs/DATA-FLOW.md`, `README`.

A2. Give a 30-second project pitch.

Spoken: Use the 30-second overview in Chapter 1, then offer one concrete trade-off: outbox durability permits duplicate dispatch after a crash. **Deeper:** Name the API transaction, publisher, consumer, state/version checks, and the at-least-once/non-exactly-once boundary. Tailor the final sentence to your actual contribution rather than claiming team-wide work.

Likely follow-up: Which part did you personally own? **Avoid:** Reciting every technology name without a problem/guarantee. **Verify:** Chapter 1 and your own commit history/notes.

B. Individual contribution and ownership

B1. What did you personally build?

Spoken: “I worked on [accurate components]. The repository shows how those components behave; I can walk through [specific source/test] and explain my decisions.”

Deeper: Separate authored code, design, debugging, tests, documentation, and prior work. This repository audit cannot determine the author's personal contribution or team context. Give a bounded example you can defend from your own history.

Likely follow-up: Which decision would you change, and why? **Avoid:** Claiming sole authorship of the entire system without evidence. **Verify:** Your own commit/PR history and files you can explain line by line.

B2. Describe an engineering challenge you faced.

Spoken: Do not invent an incident. Select a real experience you can substantiate; if using this project, present the outbox crash window as a design failure scenario, not as something you personally experienced.

Deeper: State context, observed evidence, alternatives, change, and verification. A code-backed scenario can explain the dual write and duplicate publication without pretending it happened in production.

Likely follow-up: What log or test proved your fix? **Avoid:** Converting a hypothetical failure timeline into a personal debugging story. **Verify:** `docs/TESTING.md`, relevant diff/PR, `jobs/outbox.py`.

C. Problem statement and requirements

C1. Why make inference asynchronous?

Spoken: “It separates request acceptance from unpredictable provider latency. The client receives a durable job ID and can inspect status later.”

Deeper: Asynchrony reduces coupling to HTTP/proxy timeouts and allows independent worker capacity, but adds durable state, delivery, retries, visibility and cancellation complexity. It is not automatically simpler or lower latency end-to-end.

Likely follow-up: When would a synchronous call be better? **Avoid:** Saying asynchronous means instant completion.

Verify: `jobs/service.py`, `api/v1/jobs.py`, Chapter 2.

C2. Which requirements are actually implemented?

Spoken: “Durable submission, idempotency, outbox dispatch, Kafka workers, selected retries, one-shot scheduling, auth/ownership, and status history are implemented. Recurring schedules, exactly-once execution and production deployment are not claimed.”

Deeper: Separate current code from generic needs and proposed future work. For example, durable retry intent exists, but no operator replay UI/API does.

Likely follow-up: Which missing feature blocks production first? **Avoid:** Treating an item in an architecture diagram as shipped. **Verify:** `docs/KNOWN-LIMITATIONS.md`, code and migration path.

D. Architecture and component responsibilities

D1. Walk through the architecture.

Spoken: “The API commits job plus outbox intent to PostgreSQL; a publisher sends versioned dispatch to Kafka; a worker claims and executes; the scheduler activates due jobs; the console reads persisted status.”

Deeper: Kafka is not the lifecycle source of truth. PostgreSQL owns the state machine, attempts and results. The scheduler writes due intent but doesn't publish. Redis does not participate in job correctness.

Likely follow-up: Which process can be unavailable without rejecting a valid submission? **Avoid:** Calling these modules separately deployed microservices by default. **Verify:** Compose file, app entrypoints, `docs/ARCHITECTURE.md`.

D2. Why separate publisher, scheduler, worker and API processes?

Spoken: “They have different loops and dependencies, while sharing one modular backend and database contract.”

Deeper: Process separation allows different lifecycle and scaling knobs; the trade-off is more configuration, connections, supervision and deployment coordination. It is process isolation, not evidence of independent service ownership or separate data stores.

Likely follow-up: What would you consolidate for a tiny deployment? **Avoid:** Saying microservices are inherently more scalable. **Verify:** `backend/src/aetherflow`, Compose services, ADR-0017.

E. Python and FastAPI

E1. Why FastAPI rather than Django or Flask?

Spoken: “FastAPI fits a typed async API with Pydantic validation and generated OpenAPI. I would choose based on project needs, not claim it is universally better.”

Deeper: Django REST Framework provides more integrated admin/auth and conventions; Flask can be lighter but leaves more decisions to extensions. The inspected repository does not document a head-to-head framework evaluation.

Likely follow-up: What would Django simplify here? **Avoid:** Attributing undocumented alternatives to original decision history. **Verify:** `backend/pyproject.toml`, API app/routes, ADR directory.

E2. How do async Python and SQLAlchemy fit together?

Spoken: “FastAPI route dependencies provide an async session; service functions own transactions, and provider/Kafka I/O is kept outside DB transactions.”

Deeper: An async function only helps when awaited I/O is genuinely nonblocking. A per-process connection pool must be included in replica capacity; blocking CPU work would still require separate treatment.

Likely follow-up: Can an async endpoint create unbounded concurrency? **Avoid:** Equating `async` with unlimited throughput. **Verify:** `infrastructure/database/session.py`, route dependencies, worker.

F. REST API design and validation

F1. How does the submission API validate input?

Spoken: “Pydantic schemas bound fields, reject extra fields on the request, normalize timezone-aware schedules to UTC, and reject credential-like configuration.”

Deeper: The schema is an API boundary, not a promise that every model or provider supports every shape. Domain services separately validate idempotency and state. Validation errors follow the public structured error contract.

Likely follow-up: Where would you add a per-job typed output schema? **Avoid:** Calling arbitrary dictionary output a validated business schema. **Verify:** `jobs/schemas.py`, `api/errors.py`, provider `ExecutionOutcome`.

F2. How does idempotency work?

Spoken: “The caller supplies an idempotency key. It is scoped to the principal and paired with a request fingerprint; same key/same request returns the prior job, different payload conflicts.”

Deeper: The unique database constraint handles concurrent submissions. The idempotency record and job creation are committed together. This protects HTTP submission, not provider execution or Kafka publication.

Likely follow-up: What if two same-key requests race? **Avoid:** Saying the key guarantees one provider call. **Verify:** `jobs/service.py`, `idempotency_records` model/migration/tests.

G. PostgreSQL, transactions, indexes and migrations

G1. Which consistency boundary is PostgreSQL responsible for?

Spoken: “It is the source of truth for job state, idempotency, attempts, results and the outbox. A transaction couples job acceptance to dispatch intent.”

Deeper: State/version predicates and constraints guard races; database transactions do not include Kafka or the LLM. Migrations are versioned with Alembic, and due/outbox/owner/lease indexes support their query paths.

Likely follow-up: Which writes are deliberately separate transactions? **Avoid:** Claiming a PostgreSQL transaction spans Kafka. **Verify:** `infrastructure/database/models.py`, migrations, service/outbox.

G2. Why PostgreSQL instead of MongoDB?

Spoken: “The project needs relational ownership, unique idempotency, transactional outbox writes, and conditional state transitions; PostgreSQL fits those boundaries.”

Deeper: MongoDB can support transactions and may fit document-centric schemas, but choosing a document store does not remove concurrency or dual-write problems. No repository evidence shows a MongoDB prototype or original bake-off.

Likely follow-up: What data might be document-shaped? **Avoid:** “MongoDB cannot do transactions.” **Verify:** models, constraints, migration chain and ADRs.

H. Kafka

H1. Why Kafka instead of a database-backed queue?

Spoken: “Kafka provides a durable partitioned dispatch stream and consumer groups; that can isolate API acceptance from workers. It costs a broker and operational complexity.”

Deeper: A PostgreSQL queue could be simpler at modest scale and share transaction boundaries. Kafka is justified only if replay/partition/consumer requirements outweigh DB polling. ADRs record the Kafka direction, not a universal superiority claim.

Likely follow-up: What if traffic never needs Kafka-scale throughput? **Avoid:** Citing benchmarked throughput that does not exist. **Verify:** ADR-0003/0005/0012, dispatch adapter, Compose broker.

H2. How are offsets committed?

Spoken: “Auto-commit is disabled; the worker acknowledges/commits after a durable processing decision. Failure before that can cause redelivery.”

Deeper: State/version checks and constraints make duplicate messages largely safe for durable state, but cannot make the external provider call exactly once.

Likely follow-up: What if the DB commits but offset commit fails? **Avoid:** Saying an offset equals a completed provider transaction. **Verify:** `infrastructure/kafka.py`, worker runner and tests.

I. Transactional outbox

I1. Why use an outbox?

Spoken: “It commits job state and dispatch intent atomically in PostgreSQL, so an API crash after commit cannot silently lose the intent.”

Deeper: A separate publisher leases rows, sends outside the transaction, then finalizes. This removes one lost-message window but leaves possible duplicate publication after broker acknowledgement.

Likely follow-up: Why not publish inside the DB transaction? **Avoid:** “The outbox gives exactly-once delivery.” **Verify:** `jobs/service.py`, `jobs/outbox.py`, ADR-0012.

I2. What if publisher crashes after Kafka accepts a message?

Spoken: “PostgreSQL may still show the row unpublished. After lease expiry it can be sent again; the consumer must tolerate a duplicate.”

Deeper: Kafka and PostgreSQL do not share an atomic commit. Producer idempotence does not cover a new publish attempt after the process restarts. The worker checks persisted state/version and the external inference may still execute more than once in a different crash window.

Likely follow-up: What state prevents stale work from overwriting newer work? **Avoid:** Suggesting the window can be eliminated by a longer lease. **Verify:** Outbox lease predicates, Kafka producer, worker claim.

J. Distributed systems and consistency

J1. What happens if PostgreSQL is unavailable?

Spoken: “The API cannot commit a new durable job and readiness should fail. Publisher/scheduler/worker cannot make durable decisions; Kafka messages should remain unacknowledged if processing fails before persistence.”

Deeper: Exact client error/retry behavior depends on the route/driver and runtime, so verify logs and error handlers. Existing jobs remain in Kafka or the outbox, but recovery needs PostgreSQL restored. Do not claim every process has a tested database-outage runbook.

Likely follow-up: What does the caller do after a timeout? **Avoid:** Retrying a non-idempotent POST without its original key. **Verify:** readiness, SQL session lifetime, worker exception/ack path.

J2. What guarantees does the system not provide?

Spoken: “No exactly-once provider execution, guaranteed remote cancellation, globally ordered jobs, high availability, or production SLO is claimed.”

Deeper: Dispatch is at least once. Database state transitions use version/state checks; idempotent HTTP creation is separately supported. External effects and local commits cannot be atomically coordinated.

Likely follow-up: How do you know a job is truly complete? **Avoid:** Treating Kafka acknowledgement or a provider HTTP 200 alone as durable completion. **Verify:** result persistence, terminal transition, stale-lease recovery.

K. Worker leases, retries and recovery

K1. Can a job execute more than once?

Spoken: “Yes. A worker can call the provider, crash before persisting the result, and the system cannot know whether the external work happened.”

Deeper: Kafka redelivery and outbox duplicates are distinct from execution retry. Durable state/version guards and unique result/attempt constraints protect database invariants, not the external provider side effect. A provider-supported idempotency key could narrow the risk.

Likely follow-up: What would you need for effectively-once effects? **Avoid:** “Kafka exactly once means provider exactly once.” **Verify:** worker execution order, result constraints, provider API contract.

K2. What prevents a stale worker from overwriting a newer result?

Spoken: “A lease owner/expiry plus job state/version checks and compare-and-set updates reject stale durable transitions.”

Deeper: This fences writes only where the DB predicate is applied. A provider request already sent cannot be recalled. Recovery reconciles by result presence: result means `SUCCEEDED`; no result after expired lease means `FAILED`.

Likely follow-up: Is the lease a fencing token for the provider? **Avoid:** Saying lease expiry forcibly terminates a remote request. **Verify:** claim/finalize queries and stale execution recovery.

K3. How do you avoid retry storms?

Spoken: “Retries are bounded and capped, but the current delay is deterministic and configured jitter is not applied. Jitter and provider concurrency controls are improvements.”

Deeper: Retries only cover rate-limit, timeout and transient-provider categories. Retry budget and delay are separate from outbox publication retry. Measure correlated failures and provider quota before changing policy.

Likely follow-up: Would you honor `Retry-After`? **Avoid:** Claiming exponential backoff has jitter when it does not. **Verify:** `jobs/retry.py`, `settings/schema`, tests and ADR-0015.

L. Scheduling and cancellation

L1. What if two schedulers pick the same due job?

Spoken: “The scheduler claims due rows with database locking and then uses state/version conditions; the activation event and outbox row are transactional.”

Deeper: `SKIP LOCKED`/compare-and-set are intended to avoid two successful activations. Current SQLite tests do not prove PostgreSQL multi-process lock behavior; test that with concurrent PostgreSQL transactions.

Likely follow-up: How do cancellation and due activation race? **Avoid:** Presenting tests on SQLite as a production locking proof. **Verify:** `jobs/scheduler.py`, schedule indexes, scheduling tests.

L2. What does cancellation mean?

Spoken: “For a still-accepted scheduled job, current code cancels directly. For work already in the execution path, cancellation is a request and does not guarantee the provider call stops.”

Deeper: Scheduler and cancellation race through durable state/version. Already-published records for terminal jobs are stale no-ops. A running provider may still complete and its outcome can win according to the current state transition policy.

Likely follow-up: How would you implement hard cancellation? **Avoid:** Promising remote cancellation without provider support. **Verify:** `jobs/service.py`, state machine, current regression tests.

M. Redis and rate limiting

M1. Why use Redis/Valkey?

Spoken: “Only for optional atomic fixed-window API rate-limit counters shared by API instances. It is not a job cache, queue or correctness store.”

Deeper: Auth and general API routes have separate classes; a public API key ID or client host is hashed for key construction. Redis failure is observed and fails open, which preserves availability but removes rate-limit enforcement temporarily. Valkey protocol compatibility does not mean the project tested a Valkey deployment.

Likely follow-up: Why fail open rather than fail closed? **Avoid:** Claiming Redis is required to process a job correctly.

Verify: `observability/rate_limit.py`, `infrastructure/redis.py`, ADR-0020.

N. AI provider abstraction and deterministic mocks

N1. Why abstract the provider?

Spoken: “The executor boundary separates provider-specific transport and failure normalization from job lifecycle logic and makes tests deterministic.”

Deeper: Mock is the default; an OpenAI-compatible HTTP adapter is implemented. Normalized outcomes are dictionaries with metadata, not a universal strongly typed output schema. No automatic provider fallback is implemented.

Likely follow-up: Which provider-specific behavior should remain visible? **Avoid:** Calling the interface a guarantee that providers are interchangeable. **Verify:** `jobs/execution.py`, `providers.py`, `openai.py`, provider tests.

N2. How do you test without a live LLM?

Spoken: “Use the deterministic mock for lifecycle tests and mock HTTP transport for the OpenAI-compatible adapter's request/error mapping.”

Deeper: This proves local input/output and normalization paths, not live credentials, vendor availability, current quota or real model quality. External contract tests need safe credentials and an opt-in isolated environment.

Likely follow-up: How do you validate generated structured output? **Avoid:** Saying the provider always returns correct JSON. **Verify:** provider tests and `ExecutionOutcome` validation.

O. Next.js and frontend/API integration

O1. How does the UI receive status updates?

Spoken: “The detail view polls the API about every ten seconds while the tab is visible and the job is not terminal; there is no WebSocket path.”

Deeper: It aborts requests on unmount, pauses polling when hidden, resumes on visibility, and exposes loading/error/cancel states. Polling trades freshness for simplicity and read load.

Likely follow-up: When would you switch to SSE? **Avoid:** Describing polling as real-time push. **Verify:** `components/jobs/job-detail.tsx`, API client, UI tests.

O2. Where is the access token stored?

Spoken: “The frontend client keeps the short-lived access token in memory and uses an `HttpOnly` refresh cookie with credentialed requests.”

Deeper: Refresh rotation and current-user checks are server concerns; browser state is not authorization. Verify exact cookie flags and CORS configuration for the deployed environment.

Likely follow-up: What happens after a page reload? **Avoid:** Claiming the bearer token is in local storage. **Verify:** `auth-provider.tsx`, `lib/api/client.ts`, auth routes.

P. Authentication, authorization and security

P1. How do you prevent a user from reading another user's job?

Spoken: “The authenticated principal is passed into service queries, and ordinary lookups are owner-scoped; admins use separate role-protected operations.”

Deeper: Object-level authorization must apply to detail, events, attempts and cancellation, not merely hide a UI link. Tests include private-job isolation.

Likely follow-up: How would you test IDOR? **Avoid:** Trusting a UUID's unguessability. **Verify:** `jobs/service.py`, dependencies, auth/API tests.

P2. How would you secure an exposed demo?

Spoken: “Do not expose shared credentials casually. Use a read-only provisioned demo, isolate secrets, add quotas/rate limits, TLS and monitoring, and protect provider spend.”

Deeper: Current demo is opt-in and read-only. Redis limiting fails open; public deployment requires abuse controls and an explicit secret/data policy. Email verification/password recovery are absent.

Likely follow-up: What if Redis is down? **Avoid:** Calling a shared demo identity equivalent to production onboarding. **Verify:** demo provisioning, role policy, rate limiter, CORS/settings.

Q. Testing, debugging and quality

Q1. What does the current test suite prove?

Spoken: “It proves deterministic service/API/frontend behavior through SQLite and fakes, plus a limited prior local Compose smoke path; it does not prove PostgreSQL lock races or production availability.”

Deeper: State the dated counts only with the observed environment and warnings. Fake boundaries let tests target failure classification and contracts, but real broker rebalance, cross-process Redis and restart tests remain gaps.

Likely follow-up: What integration test would you add first? **Avoid:** “All tests passing means production-ready.” **Verify:** `docs/TESTING.md`, backend tests, frontend Vitest scripts.

Q2. How would you debug a regression?

Spoken: “Reproduce with a focused test, identify which durable boundary failed, correlate request/job IDs, inspect state/version/attempt/outbox, then add a regression test.”

Deeper: Avoid inferring a cause from a dashboard alone. For asynchronous work, follow persisted evidence from API commit through publication, offset, claim, provider call and result.

Likely follow-up: Which test isolates this layer? **Avoid:** Editing retry timing before identifying the failure category. **Verify:** request logging, event/attempt APIs, test layout and source flow.

R. Docker and local development

R1. What does Compose provide?

Spoken: “PostgreSQL, Kafka, Redis, migrations and separate API, publisher, scheduler and worker processes; the Next.js dev server runs separately.”

Deeper: Health checks and environment examples aid local startup, but Compose is not a production deployment or HA broker/database. Protect local data volumes; a smoke test should not casually reset persisted state.

Likely follow-up: Which service must be ready before the API? **Avoid:** Saying the Compose stack proves cloud deployment. **Verify:** `docker-compose.yml`, `.env.example` only, README instructions.

R2. How do you handle local secrets?

Spoken: “Use the documented local environment mechanism and example template; keep actual `.env` values out of source control and logs.”

Deeper: Provider secrets are runtime config, not job payload. Never read or print a real secret to demonstrate configuration. Rotate credentials if they are exposed.

Likely follow-up: How do you know a secret is not tracked? **Avoid:** Pasting a real environment file into a terminal transcript. **Verify:** `.gitignore`, `.env.example`, settings; never open `.env`.

S. Observability and incident diagnosis

S1. How would you debug jobs stuck in `QUEUED`?

Spoken: “Follow the dispatch path: outbox row and publication state, Kafka reachability/lag, consumer assignment, worker logs, then job version and state.”

Deeper: If still `ACCEPTED`, distinguish immediate unpublished work from a future scheduled job. Metrics reset on restart and are process-local; read the durable rows and logs too.

Likely follow-up: What if Kafka is healthy but no worker owns a partition? **Avoid:** Assuming API readiness means end-to-end processing is healthy. **Verify:** job/outbox detail/admin endpoints, broker group and process logs.

S2. What does readiness actually mean?

Spoken: “API readiness checks PostgreSQL, while liveness is dependency-free. It does not certify Kafka, Redis, or the LLM.”

Deeper: A production deployment may need role-specific probes for publisher/worker/scheduler and queue progress, while avoiding a probe that makes transient provider failure restart every process.

Likely follow-up: Would you put every dependency in readiness? **Avoid:** Equating a 200 health response with a successful job. **Verify:** `api/health.py`, Compose health checks, `docs/TESTING.md`.

T. Scalability and performance

T1. How would you scale to a million jobs?

Spoken: “First benchmark the workload and backlog behavior. Then size partitions/workers/database pools, bound concurrency, index/retain hot tables, and monitor queue age and provider quota.”

Deeper: A job count alone says little: arrival rate, payload size, execution time, retention, retry rate and SLO determine capacity. Current worker loop is sequential per consumer; adding instances is limited by topic partitions and database/provider capacity. No benchmark supports a numeric claim.

Likely follow-up: What is the first likely bottleneck? **Avoid:** Giving a jobs-per-second figure without a reproducible test. **Verify:** Compose partition count, pool settings, metrics, benchmark status.

T2. What would you measure?

Spoken: “End-to-end acceptance-to-terminal latency, oldest outbox age, Kafka lag, schedule lateness, provider latency/errors, DB pool/locks, and retry rate.”

Deeper: Break latency by stage, track percentile/error distributions and resource usage, and include degraded runs. Current metrics are local and there is no benchmark or bundled scraper/alerting.

Likely follow-up: How do you prevent metric-cardinality explosions? **Avoid:** Instrumenting labels with raw user/job IDs. **Verify:** `observability/metrics.py`, current metrics exposition.

U. Technology choices and alternatives

U1. Why PostgreSQL rather than a document store?

Spoken: “The transaction and relational constraints around job ownership, idempotency, state transitions and outbox are central. PostgreSQL is a straightforward fit.”

Deeper: Document flexibility could help variable input/output payloads, but those are JSON fields inside a relational lifecycle. A storage change should follow measured access patterns, not schema fashion.

Likely follow-up: What if payloads grow very large? **Avoid:** Saying one database is always better. **Verify:** models, indexes, query paths, migration records.

U2. When would you remove Kafka?

Spoken: “If measured load and replay/consumer needs do not justify broker operations, a PostgreSQL-backed queue could reduce components.”

Deeper: Compare operational cost, transaction/locking contention, throughput, retention/replay and delivery semantics using an actual workload. No original alternative benchmark is documented.

Likely follow-up: What behavior must remain invariant after migration? **Avoid:** Treating a technology count as architecture quality. **Verify:** ADRs and dispatch interface.

V. Deployment, hosting costs and operational readiness

V1. Why is there no public deployment?

Spoken: “I can verify a local Compose setup and limited local smoke path, but not a public production deployment. I would state my actual reason directly rather than invent one.”

Deeper: Hosting full stateful infrastructure has cost, security, backup, secrets and reliability obligations; those are valid considerations, not confirmed personal motivation. Explain what deployment would require.

Likely follow-up: What evidence can you provide instead? **Avoid:** Claiming cost or time was the reason without knowing it. **Verify:** Deployment manifests, cloud histories, your actual explanation.

V2. What is needed before public exposure?

Spoken: “Managed secrets, TLS, backup/restore, durable monitoring, quotas/abuse controls, production PostgreSQL/Kafka behavior tests, and an operational replay/incident policy.”

Deeper: Also define retention/deletion, data residency, SLOs, migration strategy, cost limits and provider terms. A successful container start alone does not address these.

Likely follow-up: How would you protect the demo provider budget? **Avoid:** Calling local Compose production-ready. **Verify:** `docs/KNOWN-LIMITATIONS.md`, ADR-0017, security settings.

W. Weaknesses, limitations and lessons

W1. What is the weakest engineering decision?

Spoken: “A defensible weakness is that failure recovery is tested mostly with SQLite/fakes while PostgreSQL locking and Kafka rebalances are not verified. Another is deterministic retry backoff without applied jitter.”

Deeper: Name the specific evidence and propose a bounded improvement: PostgreSQL concurrency tests and a tested jitter/retry-after policy. Choose the weakness you understand and can defend.

Likely follow-up: Why was that trade-off accepted? **Avoid:** Calling a documented limitation “fixed” without code/test evidence. **Verify:** `docs/TESTING.md`, `docs/KNOWN-LIMITATIONS.md`, retry source.

W2. What would you redesign with another month?

Spoken: “I would add PostgreSQL/Kafka failure-injection integration coverage, test restart/rebalance recovery, add durable external observability, and reconcile documented retry policy with actual behavior.”

Deeper: Prioritize a reliability gap over adding a fashionable component. Scope the work with measurable acceptance criteria and preserve the local mock workflow.

Likely follow-up: What would you deliberately not add? **Avoid:** Promising a rewrite or a benchmark outcome before measuring. **Verify:** gap list in testing/limitations and ADR/source discrepancies.

X. Behavioral follow-ups and ownership

X1. Tell me about a disagreement or a decision you changed.

Spoken: Use a real example from your experience. This repository alone does not establish that a disagreement or personal reversal occurred.

Deeper: Explain the competing constraints, evidence, decision owner, result, and what you learned. A technical alternative comparison can be discussed as retrospective analysis, but not dressed up as a real debate.

Likely follow-up: What evidence changed your mind? **Avoid:** Fabricating a team, incident, or stakeholder. **Verify:** Your actual notes, reviews, or commit history.

X2. How do you know this project is yours to discuss?

Spoken: “I can trace the implementation, explain what I personally contributed, identify what I did not build, and show tests and limitations.”

Deeper: Interviewers value understanding and ownership more than claiming every technology. Walk through code, explain failure windows, and be clear about the unverified deployment/performance dimensions.

Likely follow-up: Which file would you open first? **Avoid:** Claiming personal implementation facts inferred only from a repository snapshot. **Verify:** Source navigation in Chapter 22 and your contribution record.

Chapter 21 — Mock Interviews

Recruiter introduction (30 seconds)

AetherFlow is a local-first distributed AI job orchestration project. It accepts authenticated, idempotent job submissions, persists job state and dispatch intent together, publishes through an outbox to Kafka, and runs work in leased workers through a provider abstraction. I focus on being precise about its guarantees: delivery is at least once, provider execution can repeat, and the project has local tests but no verified public production deployment.

Personalize the contribution sentence only with work you actually performed.

Project walkthrough (2 minutes)

A synchronous inference endpoint couples user latency and availability to the provider. AetherFlow returns a durable job ID instead. The FastAPI submission path validates a Pydantic request, authenticates the user, scopes an idempotency key, and stores the job and outbox intent in one PostgreSQL transaction. Future jobs remain accepted until their UTC due time. A separate publisher leases an outbox row, publishes a versioned dispatch to Kafka, then records publication. A crash after Kafka accepts but before that final update can duplicate a message. The worker disables Kafka auto-commit, checks job state and version, records a `RUNNING` lease and attempt, and calls a provider adapter outside the database transaction. Result/failure state is persisted; selected transient failure types receive bounded retries with durable future dispatch intent. A scheduler activates one-shot due jobs in PostgreSQL. The Next.js console reads job, event and attempt data and polls while the detail page is active. Redis is an optional fail-open API rate limiter, not a queue or job cache. The mock provider makes local execution repeatable. Current verification is strong on deterministic unit/API/UI paths and limited local Compose smoke, but does not establish PostgreSQL lock races, Kafka rebalances or production operation.

Architecture explanation (5 minutes)

Use Chapter 1's seven-step walkthrough and draw the Chapter 3 component diagram. Speak in invariants, not a list of products:

1. PostgreSQL owns job state.
2. Submission and immediate dispatch intent share one DB transaction.
3. Kafka delivers a notification at least once.
4. Worker state/version checks protect durable transitions, not provider side effects.
5. Retryable failure and retry intent commit together.
6. A result is the evidence stale-lease recovery uses to reconcile completion.
7. API readiness is not end-to-end dispatch readiness.

If asked to claim a stronger guarantee, name the missing boundary and what evidence would be required.

Senior engineer deep-dive scenario

Interviewer: “The LLM returns success. The worker process dies before the job is shown as complete. What happens?”

Model reasoning: First distinguish whether the result row committed. If the result is durable but the terminal job transition did not, stale-lease recovery can observe the result and reconcile to `SUCCEEDED`. If no result is durable, the system cannot know whether the provider performed work. The lease eventually expires and the current recovery path records failure; subsequent retry/re-delivery could repeat work depending on the path and policy. I would inspect attempt timestamps, result row, state version, lease and logs before concluding which window occurred.

Follow-up: “How would you improve this?” Make the local result/attempt/ terminal-state persistence atomic where feasible; add a provider idempotency key when supported; inject crashes at both sides of the commit; validate PostgreSQL races. Do not promise an atomic commit with an external provider.

Distributed-systems failure debugging scenario

Interviewer: “New jobs stay `ACCEPTED`; API and database look healthy.”

Model reasoning:

1. Determine whether the job has a future `schedule_at`. A future accepted job may be waiting as designed.
2. For immediate work, inspect the matching outbox row, available time, publication state, lease and attempt/error metadata.
3. Check publisher process logs/configuration and broker advertised listener from the publisher container network.
4. If the outbox says published, inspect Kafka topic/group lag, partition assignment, worker process and deserialization errors.
5. Compare message job version with current DB state. A stale/terminal no-op is not a stuck job.
6. Correlate request/job IDs and metric counters; metrics may reset at process restart. Do not rerun a job or delete a volume to make the symptom disappear.

This sequence follows evidence across durable boundaries rather than assuming the API's readiness probe covers Kafka or a worker.

Code-review scenario

Proposed change: publish to Kafka directly inside `submit_job` and delete the outbox for simplicity.

Model review: The proposed code creates a dual-write gap: if the database commits and Kafka publish fails, the job remains accepted without durable dispatch intent; if publish occurs first and DB commit fails, the worker can see a nonexistent job. Publishing inside a DB transaction does not make Kafka part of the transaction and lengthens lock/connection duration. Preserve the outbox unless requirements and failure policy intentionally change. Add a test for commit-before-publish crash and duplicate publication after broker acknowledgement.

Skeptical interviewer challenge round

“Is this exactly once?” No. Dispatch is at least once; provider effects can repeat.

“Then why use a lease?” It bounds ownership and helps recover stale durable state; it cannot fence a remote provider without provider cooperation.

“Why not skip Kafka?” A DB queue may be simpler; the current design values a separate partitioned dispatch stream, at the cost of broker operations.

“Is the demo production-safe?” It is read-only and opt-in, but a public deployment still needs abuse controls, secrets, TLS, quotas, monitoring and tested infrastructure recovery.

“Why no public demo?” State the true reason. The repository verifies local workflows, not a public deployment. Cost and secrets are considerations, not known personal history.

“What is weakest?” Cite a verifiable gap such as missing PostgreSQL lock race/Kafka rebalance testing or deterministic retries without applied jitter. Then state a concrete next test/change.

“What would you build next?” Prioritize the failure-injection integration tests and durable operational observability before adding a new broker or microservice.

Answering follow-ups without scripts

- Clarify the failure boundary the question targets.
- State what the database, broker, and provider each know at that point.
- Walk the timeline and identify durable evidence.
- Separate current implementation from proposed improvement.
- Say what test or source file would verify the answer.
- If uncertain, say what is unknown and how you would measure/inspect it.

Chapter 22 — Source-Code Navigation Guide

Paths below are repository-relative and were checked against the current workspace. Treat current source as behavior; ADRs are design records and may be older than the present implementation.

Interview topic	Start here	Then inspect
Job submission/idempotency	backend/src/aetherflow/api/v1/jobs.py	backend/src/aetherflow/jobs/service.py, jobs/schemas.py, backend/tests/test_jobs.py
State transitions	backend/src/aetherflow/jobs/state_machine.py	jobs/service.py, backend/src/aetherflow/infrastructure/database/models.py
Database persistence	backend/src/aetherflow/infrastructure/database/models.py	infrastructure/database/session.py, backend/migrations/versions/
Outbox creation/publication	backend/src/aetherflow/jobs/service.py	jobs/outbox.py, infrastructure/kafka.py, backend/tests/test_outbox.py
Kafka envelope/offset	backend/src/aetherflow/infrastructure/kafka.py	backend/tests/test_kafka.py, ADR-0011
Worker claim/execute	backend/src/aetherflow/jobs/worker.py	jobs/execution.py, backend/tests/test_phase4a.py
Retry classification	backend/src/aetherflow/jobs/retry.py	jobs/worker.py, backend/tests/test_phase6_execution_retry.py, ADR-0015
Scheduling	backend/src/aetherflow/jobs/scheduler.py	jobs/service.py, backend/migrations/versions/0006_durable_scheduling.py, backend/tests/test_phase12_scheduling.py
Provider boundary	backend/src/aetherflow/jobs/execution.py	jobs/providers.py, jobs/openai.py, backend/tests/test_phase5_providers.py, test_phase9_provider.py
Password/session/API key	backend/src/aetherflow/auth/passwords.py	auth/tokens.py, auth/service.py, api/v1/auth.py, api/v1/api_keys.py, backend/tests/test_auth.py
Role and ownership policy	backend/src/aetherflow/auth/policies.py	api/v1/admin.py, jobs/service.py, backend/tests/test_phase13_admin.py
API error contract	backend/src/aetherflow/api/errors.py	api/v1/jobs.py, backend/tests/test_app.py
Health and metrics	backend/src/aetherflow/api/health.py	backend/src/aetherflow/observability/metrics.py, backend/tests/test_phase7_observability.py
Redis rate limit	backend/src/aetherflow/observability/rate_limit.py	infrastructure/redis.py, backend/tests/test_phase11_redis.py, ADR-0020
Frontend sign-in/session	frontend/components/auth/auth-provider.tsx	frontend/lib/api/client.ts, frontend/app/login/page.tsx, frontend/tests/api-client.test.ts
Frontend submission/list/detail	frontend/app/(console)/submit/page.tsx	frontend/app/(console)/jobs/page.tsx, frontend/components/jobs/job-detail.tsx, frontend/tests/job-detail.test.tsx
Frontend demo/admin	frontend/app/demo/page.tsx	frontend/app/(console)/admin/page.tsx, frontend/tests/demo.test.tsx
Backend tests and fixtures	backend/tests/conftest.py	Test files in backend/tests/, docs/TESTING.md
Frontend tests	frontend/vitest.config.ts	frontend/tests/, frontend/package.json

Interview topic	Start here	Then inspect
Migrations	<code>backend/migrations/versions/0001_identity_access.py</code>	Revisions <code>0002_jobs_and_idempotency.py</code> through <code>0007_demo_role.py</code>
Local process topology	<code>docker-compose.yml</code>	<code>.env.example</code> , <code>README.md</code> , <code>docs/ARCHITECTURE.md</code>

Quick navigation method

When asked a question, first find the route or process endpoint, then follow the service call, then the model/transaction, then its test. For delivery questions, trace API commit → outbox claim → Kafka send/finalize → worker consume/ack → state/result commit. Read the relevant migration when a constraint or index matters. Check the test to see whether it is a fake, SQLite, live local integration, or hosted CI result.

Chapter 23 — Glossary and Final Revision Sheets

Distributed-systems glossary

Term	Meaning in this handbook
At-least-once delivery	A record may be delivered again if acknowledgement is not durably committed.
Idempotency key	Caller-provided key/fingerprint used to return one prior submission rather than create another job.
Transactional outbox	A DB row committed with business state, later relayed to a separate broker.
Consumer group	Consumers sharing topic partitions; each active partition is assigned to one member in the group.
Offset	Position in a Kafka partition; commit records consumer progress.
Lease	Time-bounded durable claim that can be recovered after expiry. It does not stop an external API.
Fencing/version check	Conditional durable update that rejects a stale state/version writer.
Backpressure	Limiting/controlling incoming work when downstream capacity is insufficient.
Poison message	A record that repeatedly cannot be processed or is invalid.
Retry storm	Correlated retries that overload an already unhealthy dependency.
Compare-and-set	Update only if the stored state/version still equals the expected value.
Dead letter	A terminally rejected/deferred record requiring inspection; it needs an explicit operational policy.
RPO / RTO	Recovery Point Objective / Recovery Time Objective; data-loss and recovery-time targets.
SLO / SLI	Service Level Objective / Indicator; reliability target and the measurement used to assess it.
IDOR	Insecure Direct Object Reference: object access is not authorized for the current principal.
CAS	Compare-and-set; in AetherFlow, a guarded state/version write.
DLQ	Dead-letter queue, a dedicated place for terminally rejected messages.
HA	High availability; continued service despite defined component failures. Not established by local Compose.
PII	Personally identifiable information; prompts and outputs may contain sensitive content.
TTL	Time to live; expiry duration for a token, lease, or cache entry.

Important state transitions

```

ACCEPTED -> QUEUED -> RUNNING -> SUCCEEDED
                        -> RETRY_SCHEDULED -> QUEUED
                        -> FAILED
ACCEPTED / QUEUED / RUNNING / RETRY_SCHEDULED -> CANCEL_REQUESTED
ACCEPTED (scheduled and not yet activated) -> CANCELLED
CANCEL_REQUESTED -> CANCELLED | SUCCEEDED | FAILED

```

`DEAD_LETTERED` is terminal in the state machine. A future schedule begins `ACCEPTED`; the scheduler activates it to `QUEUED`. State transitions are version-checked. Consult `jobs/state_machine.py` for the exact permitted edges.

Guarantees and non-guarantees

Can defend: job/outbox intent is committed atomically for immediate work; future schedules are durable in PostgreSQL; API idempotency is principal/key scoped; dispatch is at least once; stale durable transitions are guarded by state/version/lease logic; selected failure classes have bounded retry.

Cannot claim: exactly-once LLM invocation; remote cancellation; globally ordered jobs; live-provider success; tested PostgreSQL lock races; HA/cloud deployment; durable centralized metrics; measured throughput or latency.

Top 25 questions to rehearse

1. What does AetherFlow solve?
2. Why asynchronous processing?
3. What happens from submit to result?
4. Why a transactional outbox?
5. What happens after Kafka acknowledges and publisher crashes?
6. Can provider execution repeat?
7. What exactly does idempotency protect?
8. How does the worker claim a job?
9. What does a lease protect, and what does it not protect?
10. Which errors retry today?
11. How are retry storms mitigated, and what is still missing?
12. What if PostgreSQL or Kafka is down?
13. How are due jobs activated and cancelled?
14. What if two schedulers contend?
15. Why Kafka instead of PostgreSQL polling?
16. Why PostgreSQL instead of a document database?
17. Why use Redis, and what happens on its outage?
18. What does API readiness mean?
19. How are users isolated from each other's jobs?
20. How is the mock provider useful?
21. What is the weakest verified engineering area?
22. What do the tests prove and not prove?
23. How would you scale after measuring?
24. Why is public deployment not claimed?
25. What would you implement next?

One-page pre-interview cheat sheet

- **Pitch:** durable asynchronous AI jobs with outbox → Kafka → leased worker → provider adapter → persisted result.
- **Source of truth:** PostgreSQL. Kafka is dispatch transport. Redis is optional fail-open rate limiting.
- **Outbox:** job plus intent commit together; send/finalize split means duplicate publication is possible.
- **Delivery:** at least once; offsets are committed after durable decision.
- **Execution:** may repeat if provider acts before result commit.
- **Stale claim:** lease plus state/version checks; not a remote kill switch.
- **Retry:** current retryable categories are rate limit, timeout and transient provider; bounded deterministic backoff; jitter not applied.
- **Scheduler:** durable one-shot UTC timestamps; SQLite tests do not prove PostgreSQL race semantics.
- **Provider:** deterministic mock default, OpenAI-compatible adapter; no

automatic provider fallback and no universal typed output schema.

- **Security:** Argon2; short-lived access token; rotating hashed refresh session; API-key secret hashed; ownership checks; demo is read-only.
- **Operations:** liveness is local; readiness checks PostgreSQL only; metrics are process-local; no bundled dashboards/alerts/OTel.
- **Verification:** 150 backend tests, 32 frontend tests at the recorded workspace snapshot; limited prior Compose smoke; not production proof.
- **Honesty:** no public deployment, performance result, personal incident or team ownership claim unless independently confirmed.
- **Best next evidence:** PostgreSQL concurrency, Kafka rebalance/restart, cross-process Redis and fault-injection tests.

Explain-the-project honesty checklist

- ☐ Did I separate code from ADR intent and future proposal?
- ☐ Did I confirm this behavior in current source, not an old diagram?
- ☐ Did I say which tests used SQLite/fakes and which infrastructure ran?
- ☐ Did I avoid claiming exactly-once provider execution?
- ☐ Did I identify the publisher crash duplicate window?
- ☐ Did I explain that cancellation may not stop an in-flight provider?
- ☐ Did I avoid invented incidents, team size, traffic and benchmarks?
- ☐ Did I state my personal contribution accurately?
- ☐ Did I avoid exposing credentials or private job payloads?
- ☐ Did I describe deployment status and my real reason truthfully?
- ☐ Can I open the route, service, model, migration and test for each claim?

Source-verification note

Project-specific statements in this handbook were checked against the local repository source, schemas, migrations, tests, README, architecture/API/ data-flow/limitations/testing documentation, Compose configuration and ADRs. The observed verification snapshot is dated 2026-10-07 and includes uncommitted working-tree changes. Tests and source can change after this snapshot. Generic distributed-systems explanations are educational context; recommendations are proposals, not implemented behavior. No private environment file or credential is a source for this guide.